Adam ABIDERRAHMANE **8 Juin 2026** **RT1 B1**

Maxence DABOIT

Projet Kaban RT



Année universitaire : 2025-2026

Date de rendu : 16 Juin 2026

Sommaire

Présentation du projet et description du projet : 2
Répartition des rôles : 3
Schéma relationnel de la BDD : 4
Table Profiles : 4
Table Board : 4
Table Categories : 5
Table Tasks : 5
Table Pomodoro_sessions : 5
Voici le schéma entité-association : 6
Fonctionnalité libre : 7
Architecture de l'app 11
Fonctionnalités implémentées : 11
Difficultés & solutions : 19
Conclusion 21
Amélioration effectué (VOIR TRAVAIL DE MAXENCE) : 21
Idée de proposition/d'amélioration : 21
Outils de collaboration utilisé : 21
Nos réseaux collaboratifs/là ou on a travailler : 21

Présentation du projet et description du projet :

En binôme on devait crée une application web pour booster notre productivité, de ce fait on a utilisé React pour le frontend et Supabase pour le backend. Ce projet devait mettre en pratique des notions utilisé en entreprise en développement Web moderne (gestion de routes, authentification, manipulation de BDD, sécurisation d'accès et création d'un utilisateur moderne).

Cette application qu'on construit est sous forme de tableau de bord et s'appelle KabanRT qui permet à un utilisateur connecté de gérer les tâches, les profils des utilisateurs (que les utilisateurs s'y connecte à leurs compte et s'inscrire) ainsi que les différentes sessions de travail via notre fonctionnalité particulière le timer pomodoro, notre tableau est divisé en quatre colonnes « a faire », « en cours », « validation » et « terminée ».

Toutes tâche à un titre, une description, un statuts et même une priorité elle peut être soit basse, soit haute ou soit moyenne ça peut dépendre de la catégorie ou même de la deadline (la date).

Application utilisé :

Outil utilisé	Rôle de l'outil utilisé
Github	Plateforme collaborative
React router	Navigation entre les pages
Supabase	Backend BDD (postgr SQL)

Vercel	Hébergement de l'application
Rescend	Envoie d'email transactionnels
React + Vite	Interface utilisateurs

Répartition des rôles :

Le travail a été réparti de manière équilibrée en binôme, on s'est occupé d'un domaine différents mais qui se complète, comme le projet est assez long on a répartie ça sous forme de tableau afin d'avoir des tâches fixées et surtout qui se complète voici notre plan de travail en binôme et ensuite à inversé les rôles à chaque TD (lors des quatre TD) afin de toucher à tout et d'être opérationnel si un soucis se présentais lors du projet (débugger quelque chose par exemple) :

Code couleurs :

Adam = vert

Maxence = rouge

TD 1 :

Configuration projet React et de Vite + Déploiement Github et Vercel

TD2 :

Page LoginPage.jsx (TD2) + DashboardPage.jsx et tous les onglets (TD2) + Composant UserTable.jsx (TD2)

TD 3 :

Modèle BDD : tables boards, categories, tasks (TD3) + Données de test SQL (TD3) + Composant TaskCard.jsx (TD3) + Composant TaskForm.jsx (TD3) + Composant TaskList.jsx (TD3)

TD 4 :

Composant Navbar.jsx (TD4) + ProfilePage.jsx + upload avatar (TD4) + API Route Vercel send-email.js (TD4)

Tâche réalisé	Travail de Adam	Travail de Maxence
Configuration projet React et de Vite	X	
Déploiement GitHub + Vercel	X	
Configuration Supabase et les clés API		X
Page LoginPage.jsx	X	
DashboardPage.jsx et tous les onglets	X	X
Composant UserTable.jsx		X

Modèle BDD : tables boards, categories, tasks		X
Données de test SQL		X
Composant TaskCard.jsx	X	
Composant TaskForm.jsx	X	
Composant TaskList.jsx	X	
Composant Navbar.jsx		X
ProfilePage.jsx + upload avatar	X	
API Route Vercel envoie d'email	X	X
Fonctionnalité libre : Pomodoro	X	X (bcp plus développé)
Tests et vérification (déploiement final)	X	X

Schéma relationnel de la BDD :

Pour cette partie-là on a créé cinq tables (boards, categories, tasks, profiles) sur Supabase (PostgreSQL), déjà on va montrer les cinq tables une par une :

Table Profiles :

Colonne	Type	Contrainte	Description
id	uuid	🔑 PK gen_random_uuid()	Identifiant unique (clé primaire)
email	text	(vide)	Email de l'utilisateur
Full_name	text	(vide)	Nom complet
role	text	'member'	admin / member
Created_at	timestamptz	now()	Date de création automatique

Table Board :

Colonne	Type	Contrainte	Description
id	uuid	🔑 PK gen_random_uuid()	Identifiant unique (clé primaire)
title	text	(NOT NULL)	Nom du tableau
Owner_id	uuid	FK -> auth.users.id	Propriétaire du tableau
created_at	timestamptz	now	Date de création

Table Categories :

Colonne	Type	Contrainte	Description
id	uuid	🔑 PK gen_random_uuid()	Identifiant unique (clé primaire)
name	text	'UNIQUE, NOT NULL'::text	Nom catégorie
color	text	''::text	Couleur affichage

Table Tasks :

Colonne	Type	Contrainte	Description
id	uuid	🔑 PK gen_random_uuid()	Identifiant unique (clé primaire)
title	text	(NOT NULL)	Titre de la tâche obligatoire
Description	text	(vide)	Description détaillé
status	text	'todo'	todo | in_progress | review | done
priority	text	'low'	low | medium | high
Due_date	date	(vide)	Date d'échéance
created_at	timestamptz	now()	Horodatage automa que
category_id	uuid	FK -> categories.id	Catégorie op onnelle
created_by	uuid	FK → auth.users.id	Créateur de la tâche
board_id	uuid	FK → boards.id	Tableau Kanban associé

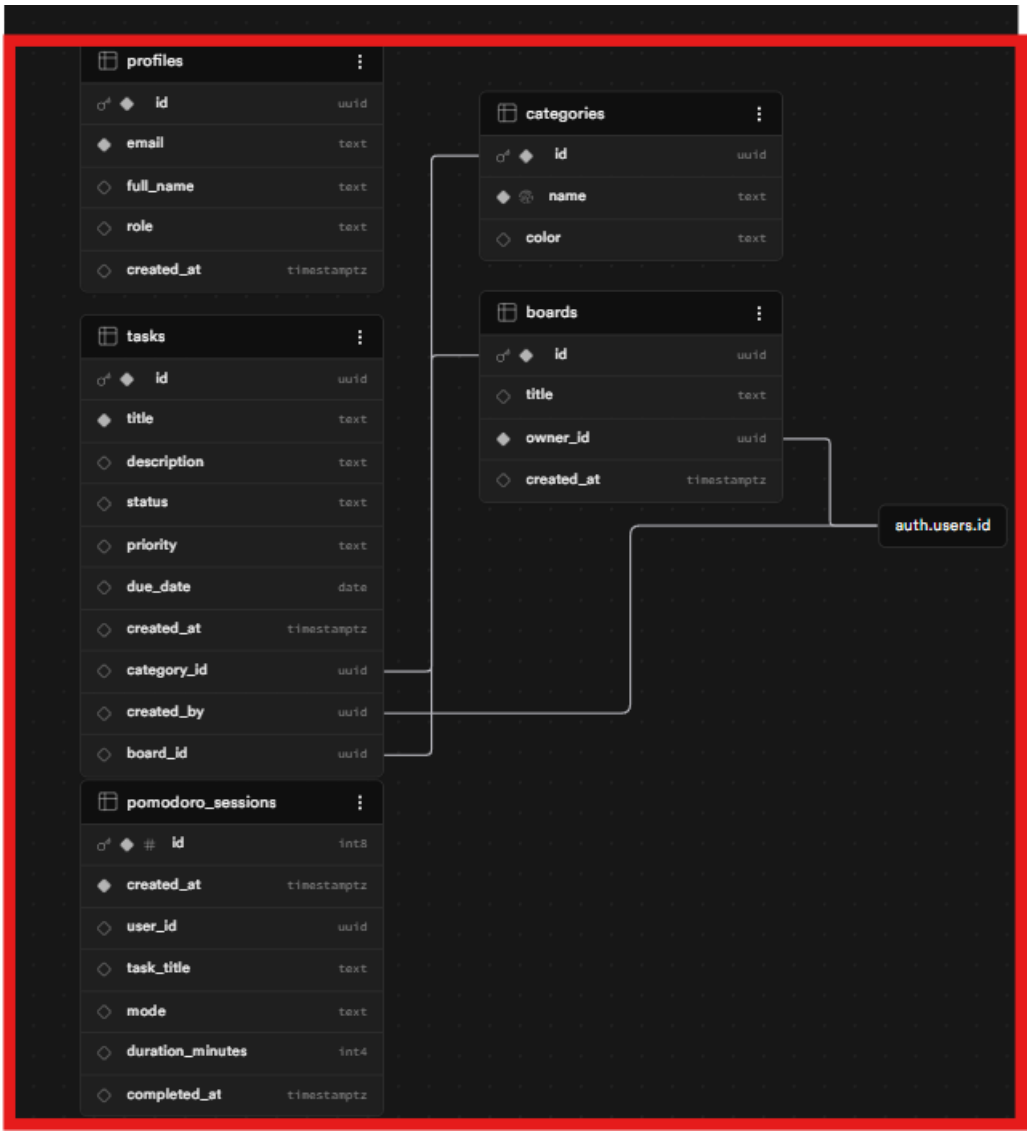
Table Pomodoro_sessions :

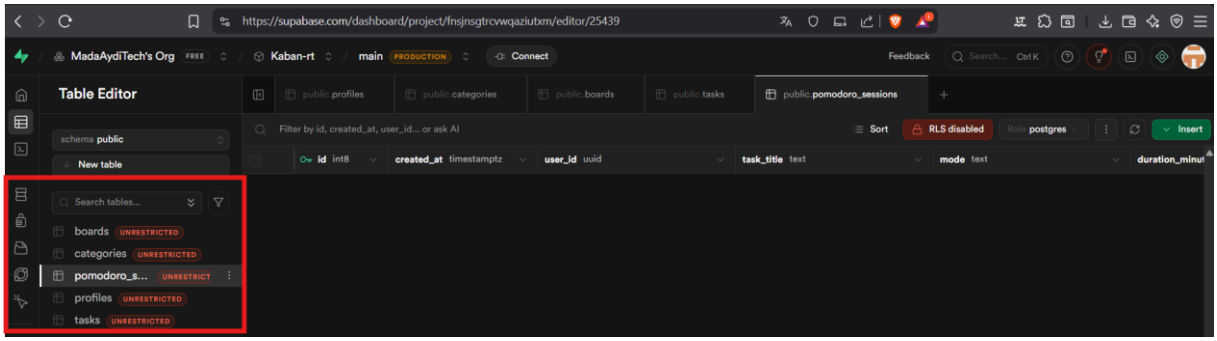
Colonne	Type	Contrainte	Description
id	Int8	🔑 PK gen_random_uuid()	Identifiant unique (clé primaire)
Created_at	timestamptz	Now()	Utilisateur concerné
User_id	uuid	Gen_random_uuid	Tâche associée

Task title	text		Titre de la tâche (sauvegardé en texte ou historique)
Mode	text		Mode de la session ('focus', 'short_break', 'long_break')
Duration_minutes	Int4		Durée en minutes
Completed_at	timestamptz	Now()	Sessions complété

Voici le schéma entité-association :







Fonctionnalité libre :

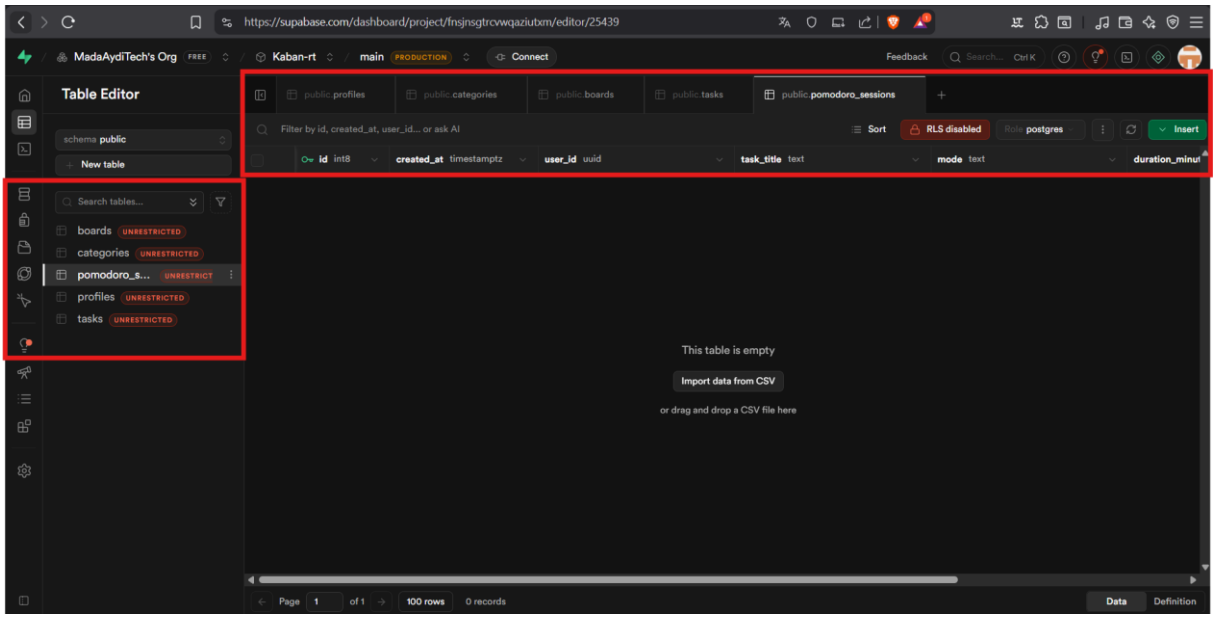
Le besoin :

La fonctionnalité qu'on a choisi de prendre c'est le timer pomodoro comme c'est un kanban on a donc une application de gestions de tâches, la concentration est importante tout autant que l'organisation. La méthode Pomodoro c'est le fait de travailler 25min sans distraction, interruption puis de prendre

une courte pause de 5min voir 7min, on a mis en place ce système afin que l'utilisateur puisse lancer un minuteur lancé à une tâche spécifique et voir son historique de concentration par la suite !

Pour réaliser ce besoin on a crée une nouvelle table pomodoro_sessions au schéma (on le voit juste au dessus) via Supabase, cette table permet d'enregistrer chaque session de travail avec sa durée ainsi que son statut c'est-à-dire si la tâche à été réussi ou pas fin complétée avec l'utilisateur concerné bien évidement.

On a crée le composant react PomodoroTimer.jsx au sein du Dashboard, l'utilisateur mettre un minuteur en lançant un compte à rebours de 25minutes puis il fait démarrer pour commencer sa tâche :



Ensuite on a fait réalisé la page PomodoroTimer.jsx :

Pour ça, on a mis en place les différentes heures

const [workMinutes, setWorkMinutes] = useState(25); c'est le temps de focus

const [shortBreakMinutes, setShortBreakMinutes] = useState(5); le temps de pause courte

const [longBreakMinutes, setLongBreakMinutes] = useState(15); le temps de pause longue

On peut changer les différentes sections de temps via les inputs, c'est quand on change workminutes bah la session change niveau timing on peut le temps de session :

const [mode, setMode] = useState('work'); choisit le mode (focus, pause courte, pause longue)

const [timeLeft, setTimeLeft] = useState(25 * 60); On démarre à 1500 secondes car 25min par minutes

const [isRunning, setIsRunning] = useState(false); Est-ce que le minuteur à démarrer oui ou non

const [completedWorkSessions, setCompletedWorkSessions] = useState(0); ici ça compte le nombre de session de travail effectué

Ensuite on y met le bruit, le moteur :

Si le timer est arrêter ou que le temps est écoulé on sort du code et on a un return, c'est le nettoyage donc si on clique sur pause on annule le timeout en court directement si on ferait pas ça on aurait eu plein de timeout qui s'empilent avec le timer qui s'accélère.

```
useEffect(() => {
  if (!isRunning || timeLeft <= 0) return;

  const timeout = setTimeout(() => {
    setTimeLeft((prev) => prev - 1);
  }, 1000);

  return () => clearTimeout(timeout);
}, [isRunning, timeLeft]);
```

Ici on a le deuxième use effect donc c'est la sorte de cerveau qui va décider quoi faire, ça se déclenche uniquement lorsque timeLeft est à 0. Différents scénario sont possible soit l'utilisateur vient de finir sa session de travail avec **mode === 'work '** donc soit on incrémente une session **nextCount = completedWorkSessions + 1** et donc tu regarde si tu passe en pauses longue ou courte et après le timer se stop avec une action de l'utilisateur.

Le second scénario possible c'est qu'on vient de finir notre pause peu importe la pause (courte longue), on repasse en mode travail puis une fois finis le temps se stop.

React surveille ces valeurs. Dès que timeLeft change (et qu'il vaut 0), ce code s'exécute. Tu mets aussi mode et les minutes dans les dépendances parce que tu les utilises à l'intérieur. C'est la règle des hooks c'est-à-dire que tout ce que tu lis dans un useEffect doit être dans le tableau de dépendances.

```
useEffect(() => {
  if (timeLeft > 0) return;  // Si le temps n'est pas fini, on ne fait rien

  if (mode === 'work') {
    const nextCount = completedWorkSessions + 1;
    setCompletedWorkSessions(nextCount);

    if (nextCount % 4 === 0) {
      setMode('long');
      setTimeLeft(longBreakMinutes * 60);
    } else {
```

```
    setMode('short');
    setTimeLeft(shortBreakMinutes * 60);
   }
  } else {
   setMode('work');
   setTimeLeft(workMinutes * 60);
  }

  setIsRunning(false);
}, [timeLeft, mode, completedWorkSessions, workMinutes, shortBreakMinutes, longBreakMinutes]);
```

Ensuite on a les fonctions utilitaires donc on donne le temps avec fin les minutes entières via **Math.floor(seconds / 60),** les secondes qui reste **seconds % 60** et juste on ajoute un 0 si c'est 5min bah ça va devenir 05min en affichage que si c'est un seul chiffre via **padStart(2, '0')**

```
function formatTime(seconds) {
  const mins = String(Math.floor(seconds / 60)).padStart(2, '0');
  const secs = String(seconds % 60).padStart(2, '0');
  return `${mins}:${secs}`;
}
```

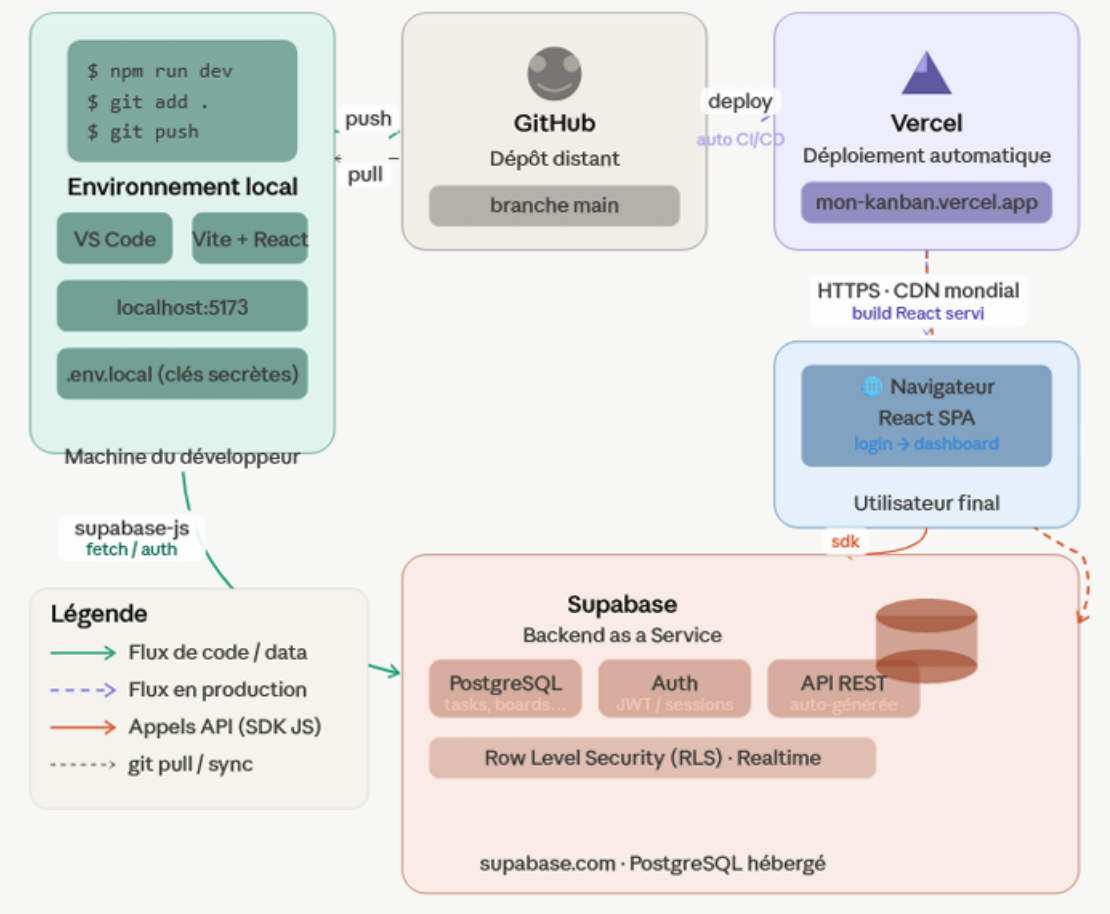
Et juste après on a la fonction reset à côté du bouton pause :

```
function resetTimer() {
  setIsRunning(false);
  if (mode === 'work') setTimeLeft(workMinutes * 60);
  if (mode === 'short') setTimeLeft(shortBreakMinutes * 60);
  if (mode === 'long') setTimeLeft(longBreakMinutes * 60);
}
```

Et pour finir le reste du code c'est les boutons quasiment (**voir dans pomodoroTimer.jsx**) mais globalement le code est compris.

Architecture de l'app

L'architecture de l'application est sur quatre plateformes qui sont interconnectées. En tant que développeur on travail en local via Vite, github et à chaque push on redéploie automatiquement le tout sur Vercel c'est l'automatisation de tâches. Supabase est le côté backend qui sert à héberger les données fin la base de données BDD (PostgreSQL) ainsi que le système d'authentification voici un schéma pour mieux comprendre ce que nous vous expliquons :



Fonctionnalités implémentées :

.env.local : Les clés secrètes pour parler à Supabase. Caché dans. gitignore pour ne jamais finir public sur GitHub.

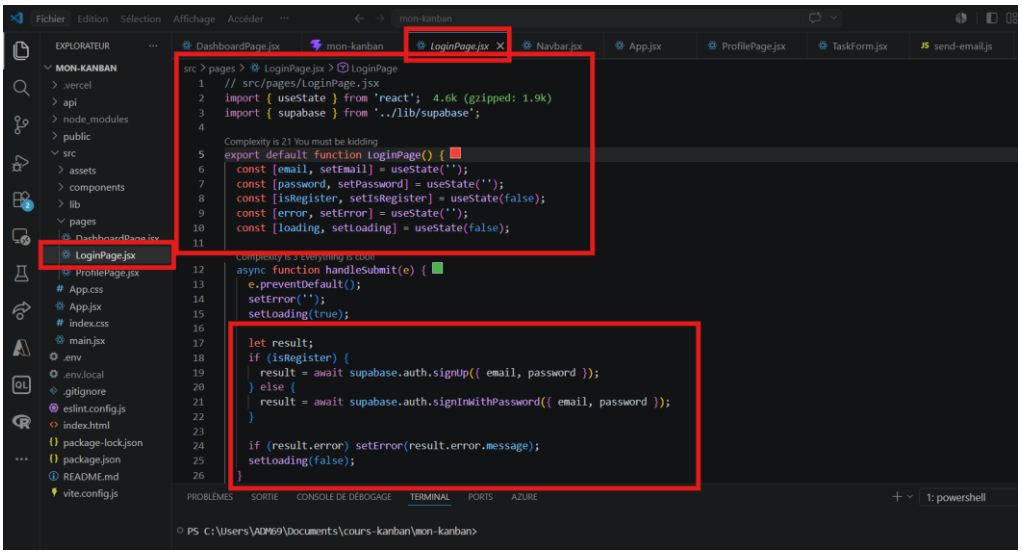
supabase.js : La passerelle. Il initialise le client Supabase avec les clés du .env pour que tout le projet puisse interagir avec la BDD.

DashboardPage.jsx : La page maîtresse. Elle gère le menu par onglets pour afficher soit le Kanban, soit le Pomodoro, soit les utilisateurs.

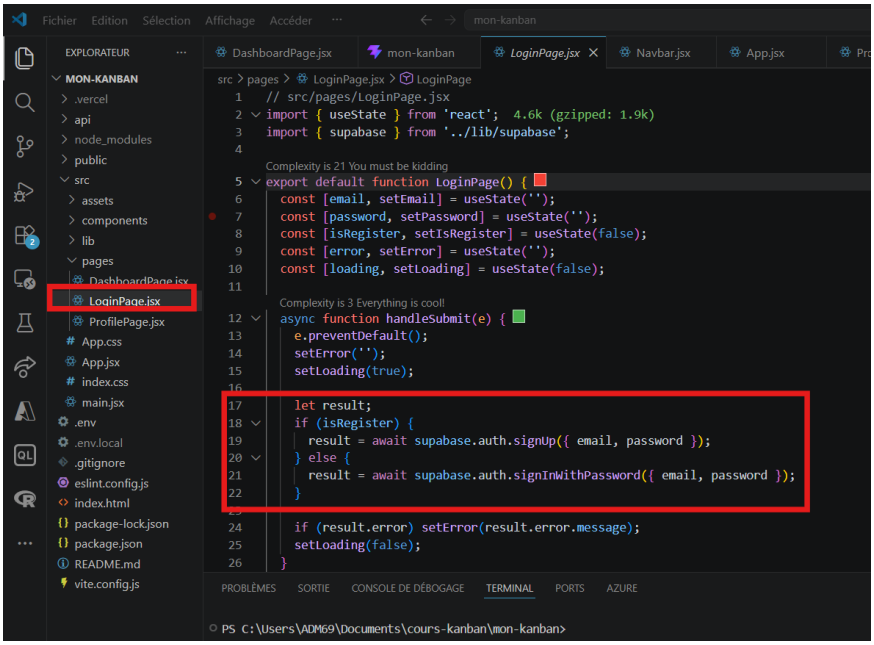
TaskForm.jsx : Formulaire de création. Quand on valide, il fait un INSERT en base de données pour enregistrer la nouvelle tâche.

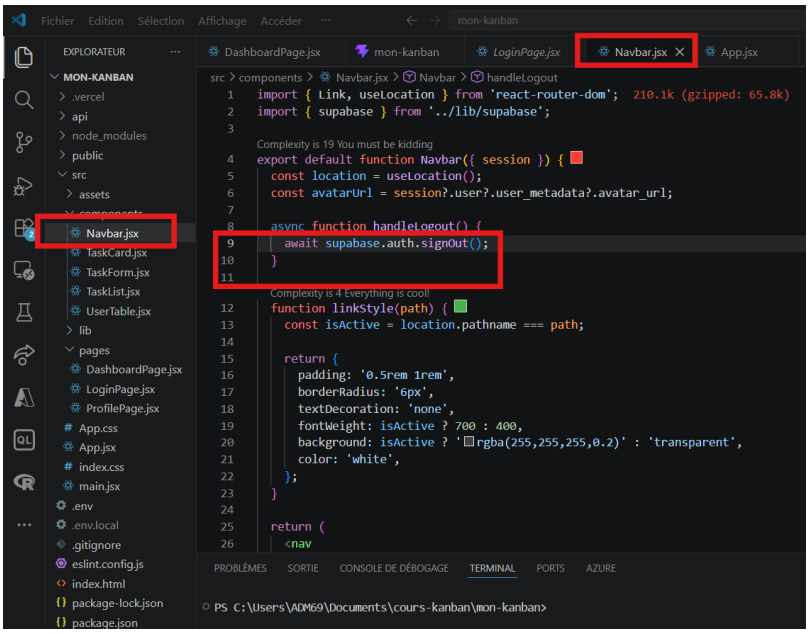
Pour réaliser ce projet on a mis en place plusieurs fonctionnalités, la première était de mettre en place l'inscription email avec les mots de passe au sein de la page **LoginPage.jsx** la requête qui m'a principalement permis de réaliser cette fonctionalité est supabase.auth.signUp():

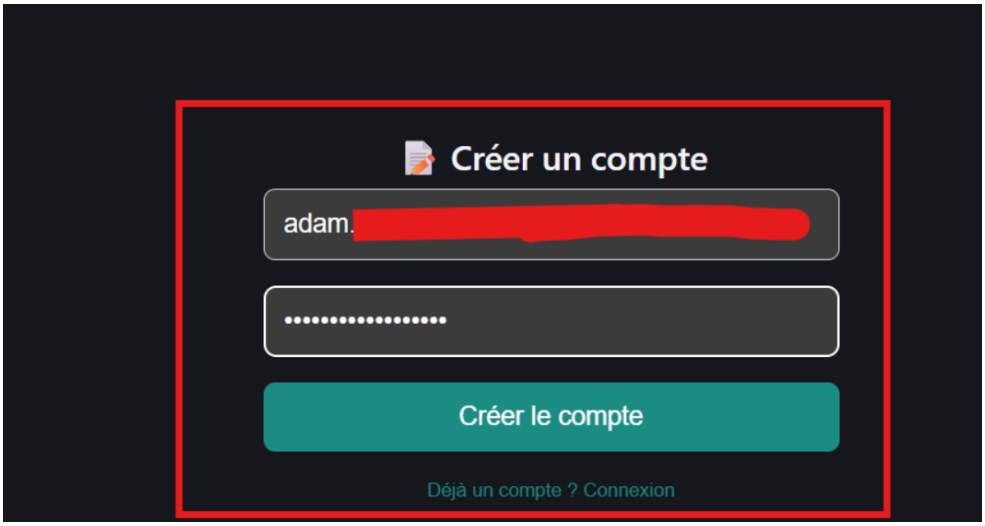
LoginPage.jsx : Formulaire d'inscription/connexion. Une fois l'utilisateur authentifié par Supabase, il switch vers le dashboard.

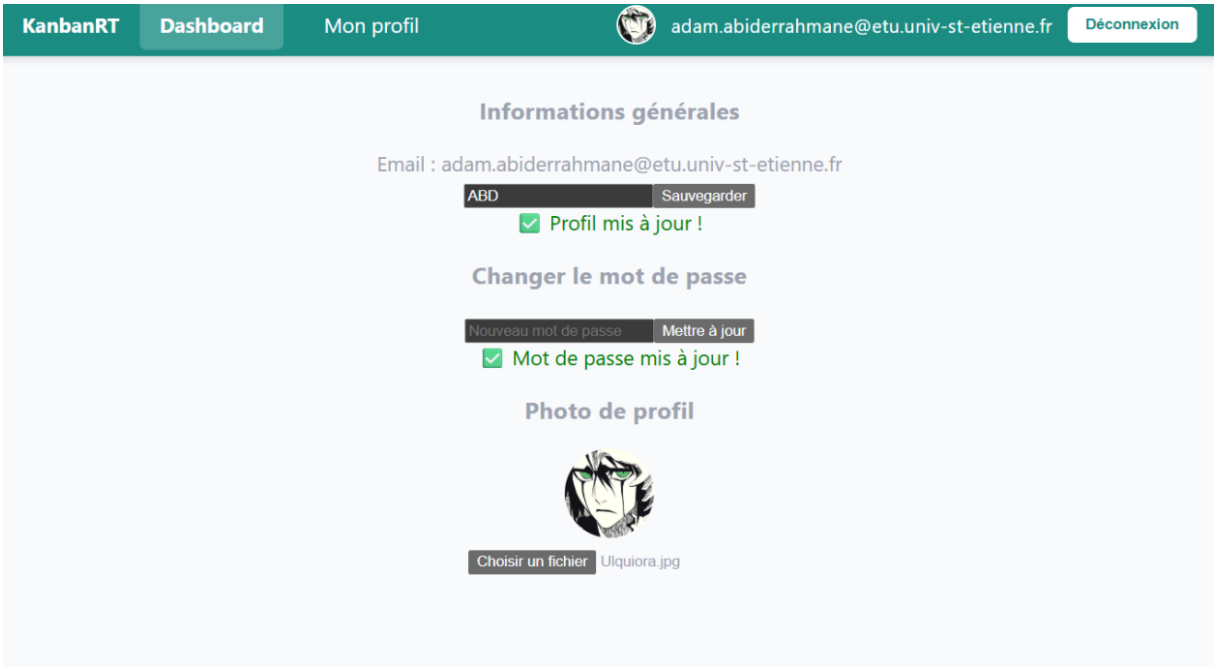


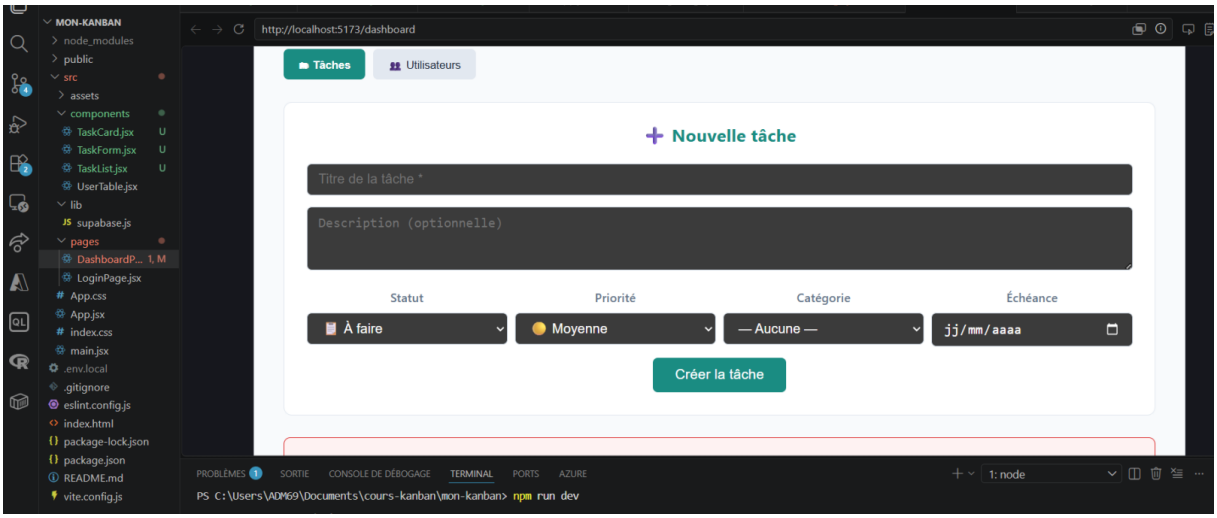
La seconde fonctionnalité qu'on a mis en place c'était la connexion et la déconnexion de l'utilisateur via les pages **LoginPage.jsx** et **Navbar.jsx** (signInWithPassword() et SignOut()):

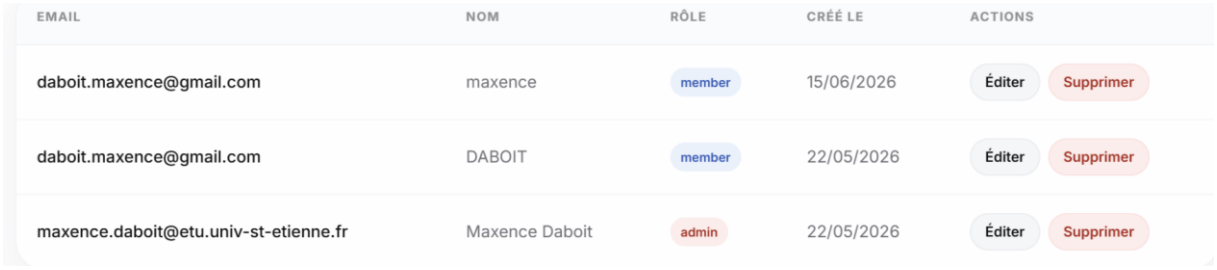


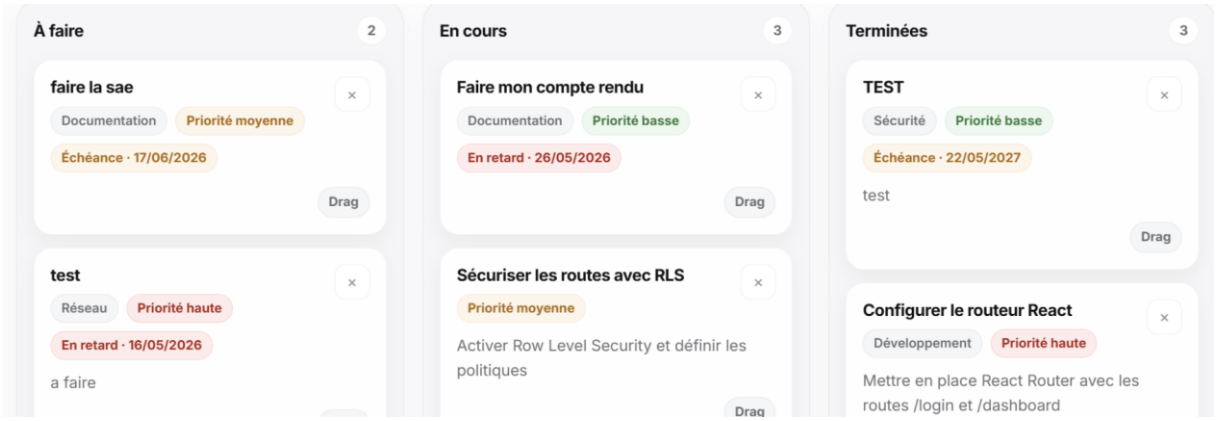

Fichier Edition Sélection Affichage Accéder
mon-kanban
EXPLORATEUR
MON-KANBAN
.vercel
api
node_modules
public
src
assets
Navbar.jsx
TaskCard.jsx
TaskForm.jsx
TaskList.jsx
UserTable.jsx
lib
pages
DashboardPage.jsx
LoginPage.jsx
ProfilePage.jsx
App.css
App.jsx
index.css
main.jsx
.env
.env.local
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
DashboardPage.jsx
mon-kanban
LoginPage.jsx
Navbar.jsx
App.jsx
src > components > Navbar.jsx > Navbar > handleLogout
import { Link, useLocation } from 'react-router-dom'; 210.1k (gzipped: 65.8k)
import { supabase } from '../lib/supabase';
Complexity is 19 You must be kidding
export default function Navbar({ session }) {
const location = useLocation();
const avatarUrl = session?.user?.user_metadata?.avatar_url;
async function handleLogout() {
await supabase.auth.signOut();
}
Complexity is 4 Everything is cool!
function linkStyle(path) {
const isActive = location.pathname === path;
return {
padding: '0.5rem 1rem',
borderRadius: '6px',
textDecoration: 'none',
fontWeight: isActive ? 700 : 400,
background: isActive ? 'rgba(255,255,255,0.2)' : 'transparent',
color: 'white',
};
}
return (
<nav
PROBLÈMES SORTIE CONSOLE DE DÉBOGAGE TERMINAL PORTS AZURE
PS C:\Users\ADM69\Documents\cours-kanban\mon-kanban>

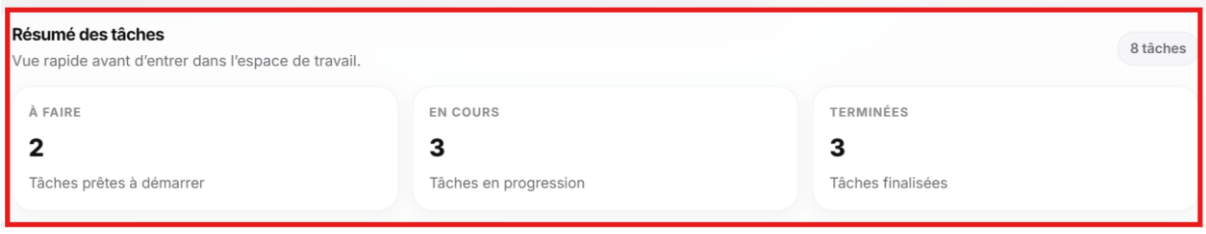

Créer un compte
adam.
Créer le compte
Déjà un compte ? Connexion


KanbanRT
Dashboard
Mon profil
adam.abiderrahmane@etu.univ-st-etienne.fr
Déconnexion
Informations générales
Email : adam.abiderrahmane@etu.univ-st-etienne.fr
ABD
Sauvegarder
Profil mis à jour !
Changer le mot de passe
Nouveau mot de passe
Mettre à jour
Mot de passe mis à jour !
Photo de profil
Choisir un fichier
Ulquiora.jpg



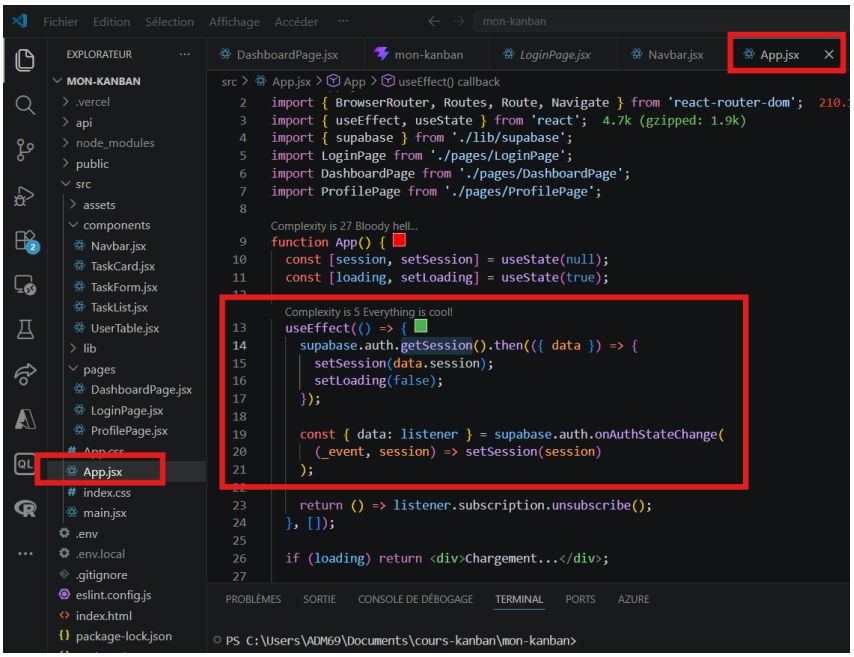






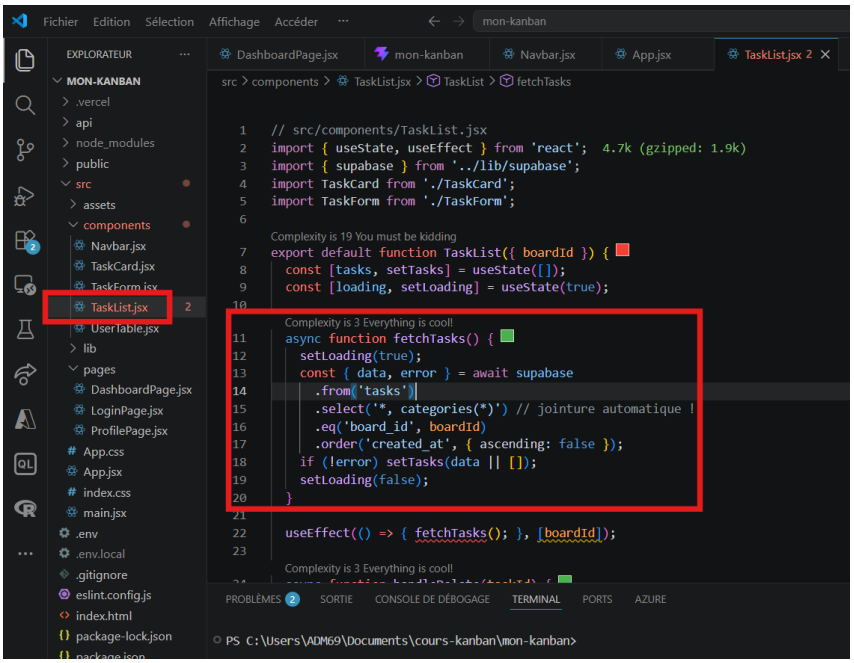
Ensuite c'est le tableau de bord du compte via la page App.jsx, on crée des routes via cette page qui sont protégé (getSession() et onAuthStateChange()).

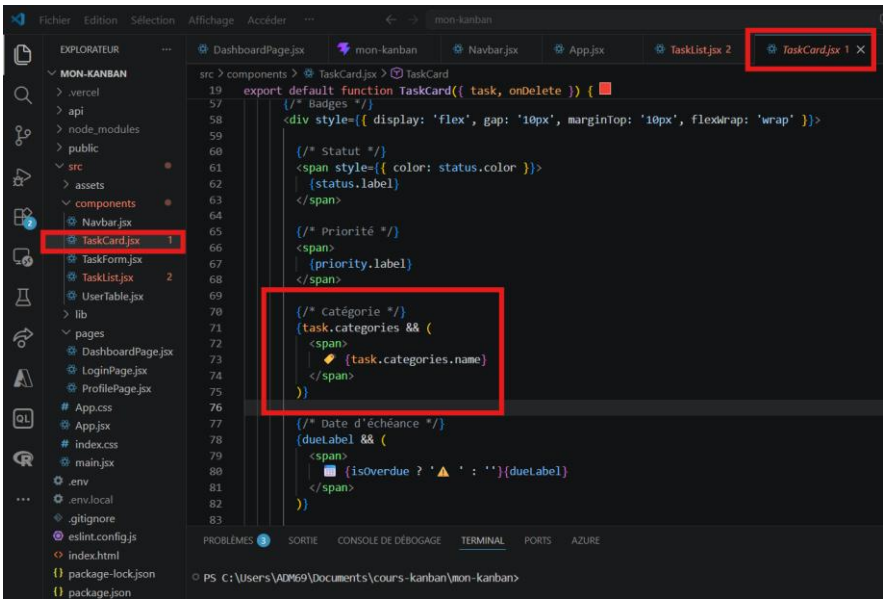
App.jsx : Le routeur sécurisé. C'est lui qui bloque l'accès au /dashboard et te redirige vers le login si tu n'es pas connecté.



Après les routes on a mis en place l'affichage des tâches TaskList().jsx et la page TaskCard.jsx (.from 'task' et .select('*, categories(*)'))

Puis la création de tâche via le formulaire sur la page Task .jsx avec .from('tasks').insert([...]) :

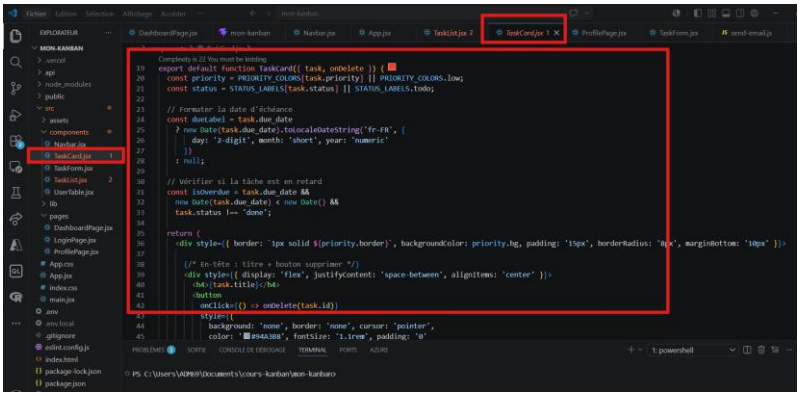




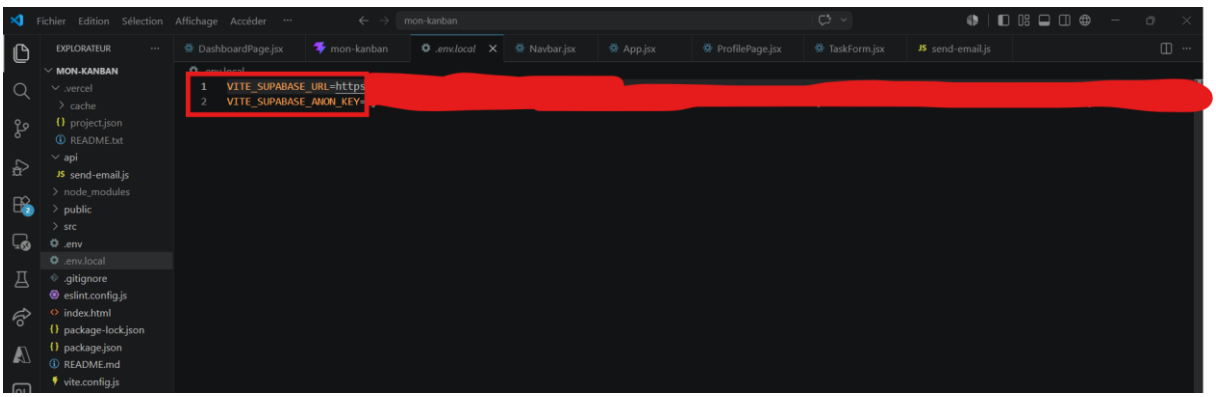
Bien évidement si on crée une page on doit aussi la supprimer donc on a mis en place une fonctionnalité qui nous permet de supprimer une tâche avec TaskCard.jsx et TaskList.jsx avec les commandes .from('tasks') .delete().eq('id', id) :

TaskCard.jsx : Le design d'une tâche. Le petit rectangle blanc visuel qui affiche le titre, la description et le badge de priorité coloré.

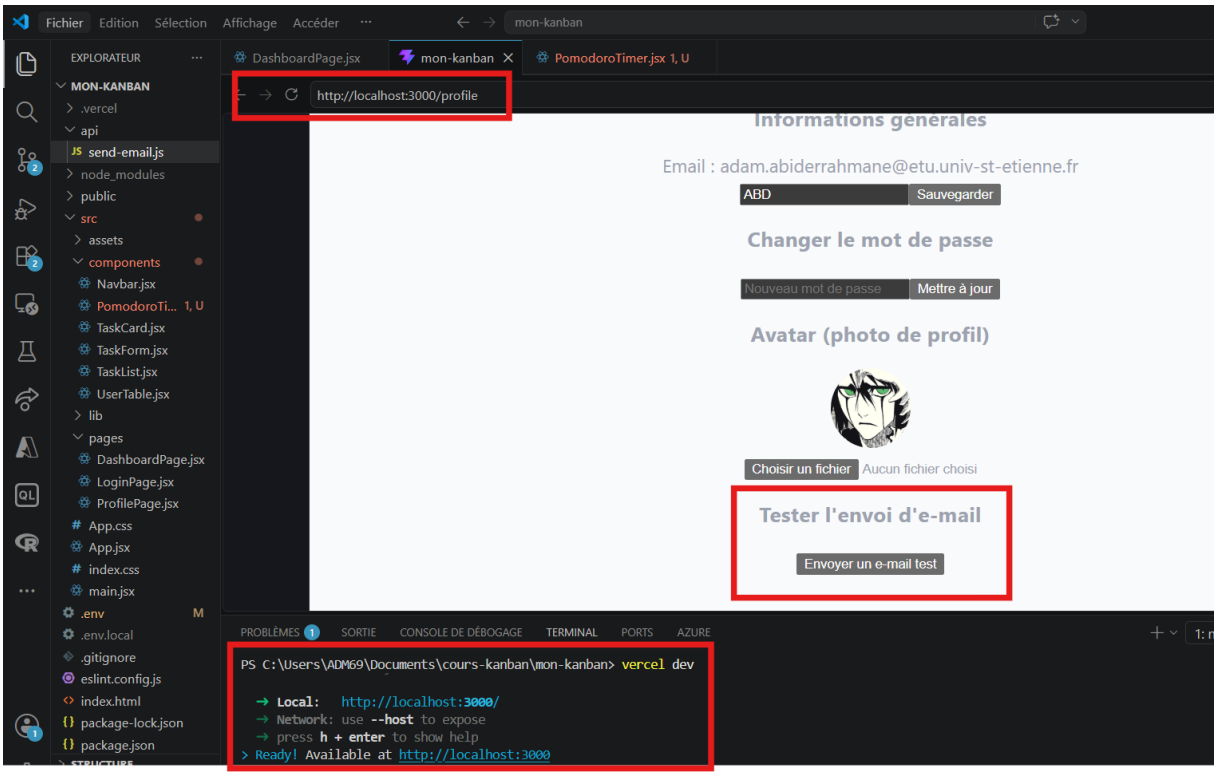
TaskList.jsx : Le cerveau du Kanban. Il va chercher les tâches sur Supabase, intègre le formulaire (TaskForm) et génère les cartes (TaskCard).

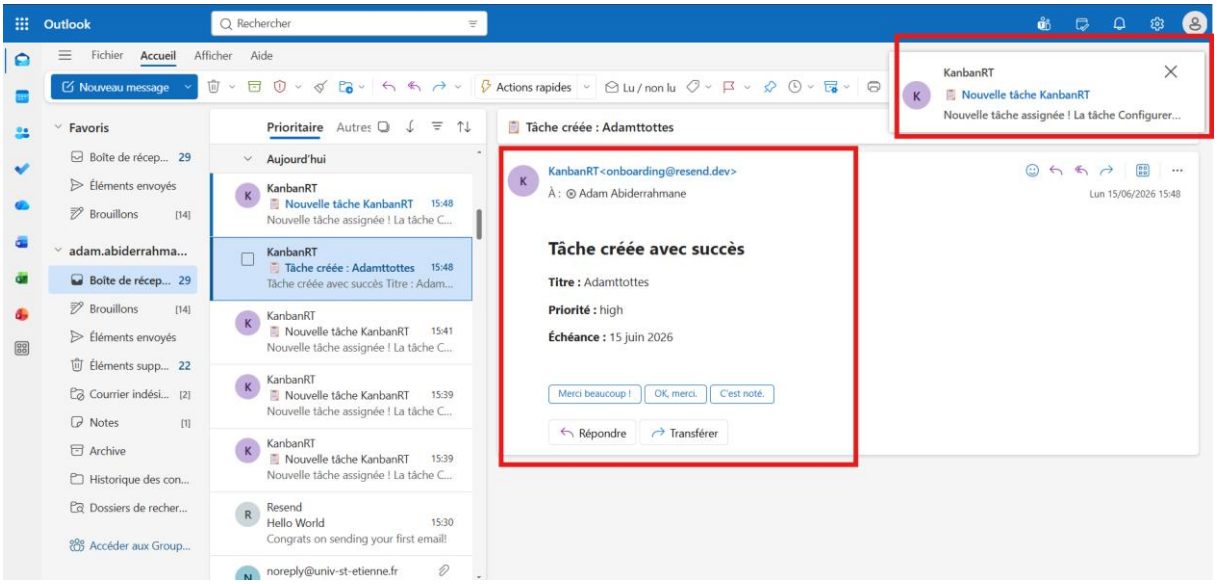


Et enfin le déploiement de l'application via Vercel avec la variable VITE_SUPABASE_URL et VITE_SUPABASE_ANON_KEY :

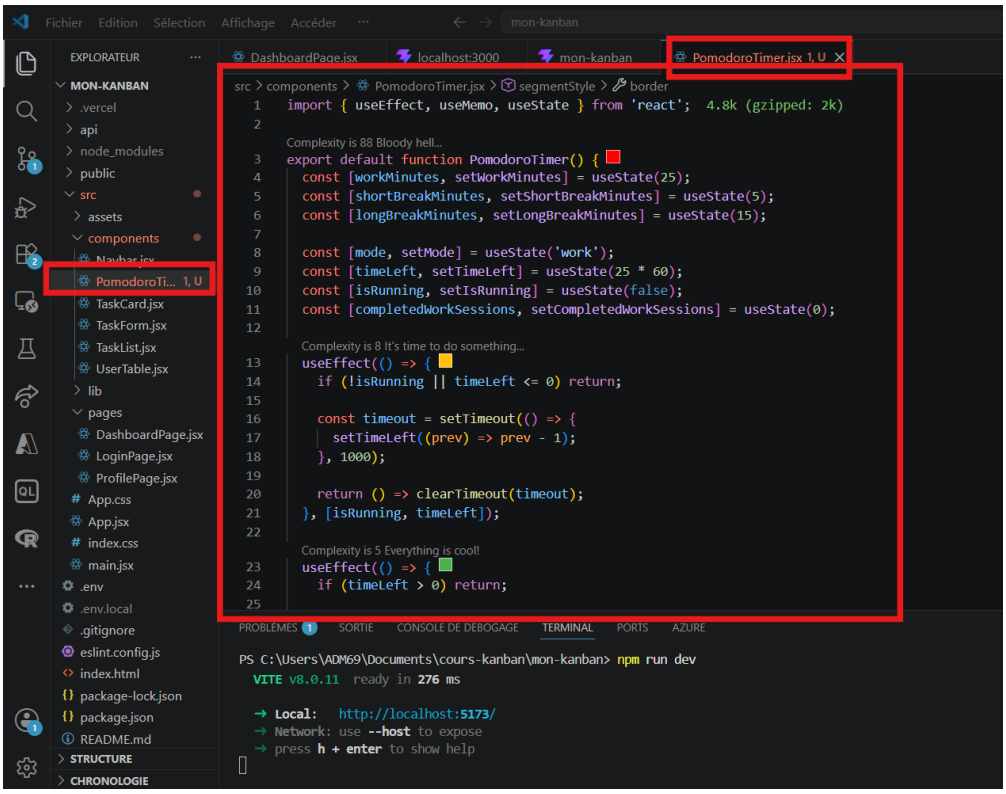


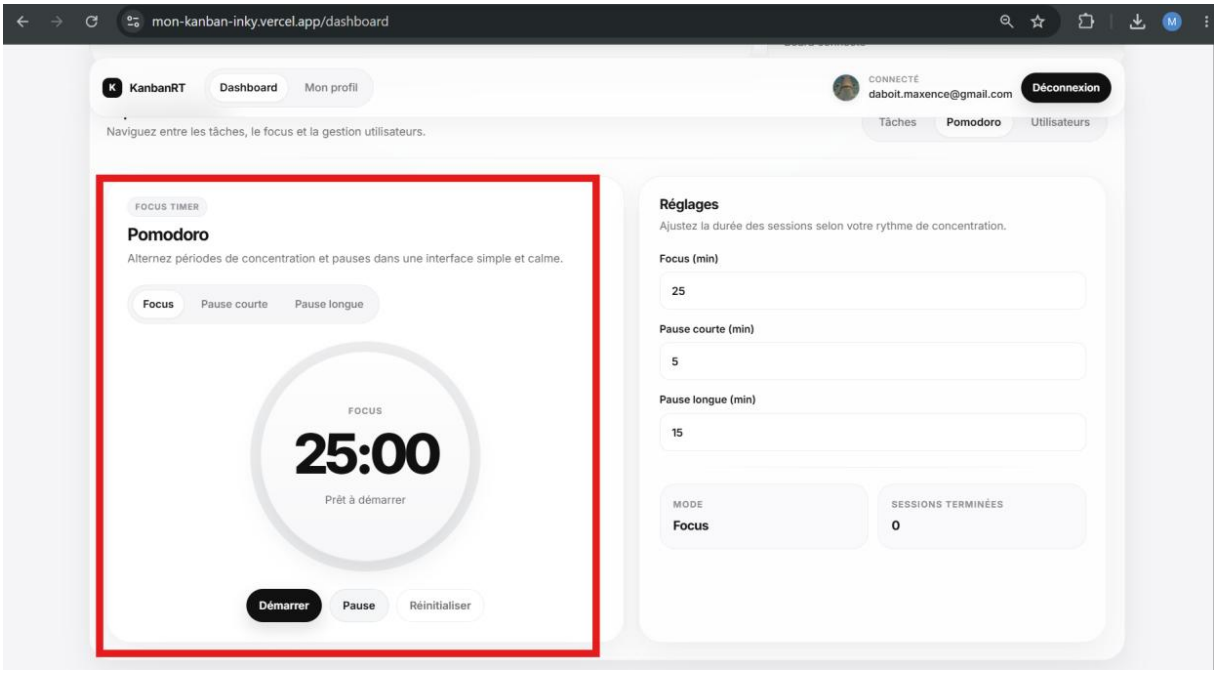
On a même mis en place lors des TD une fonctionnalité qui nous permet d'envoyer des messages mais cette fonctionnalité fonctionne chez moi (Adam) que quand je lance mon kanban via la commande **vercel dev** :

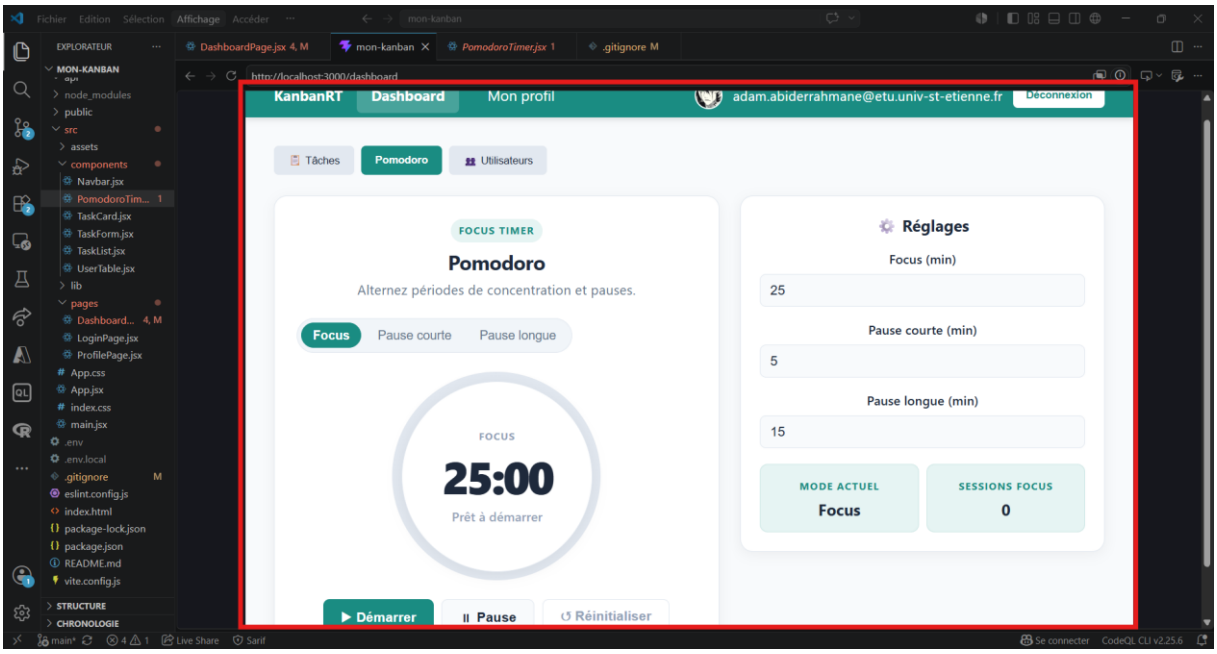




En dernier on a ajouté le minuteur pomodoro :





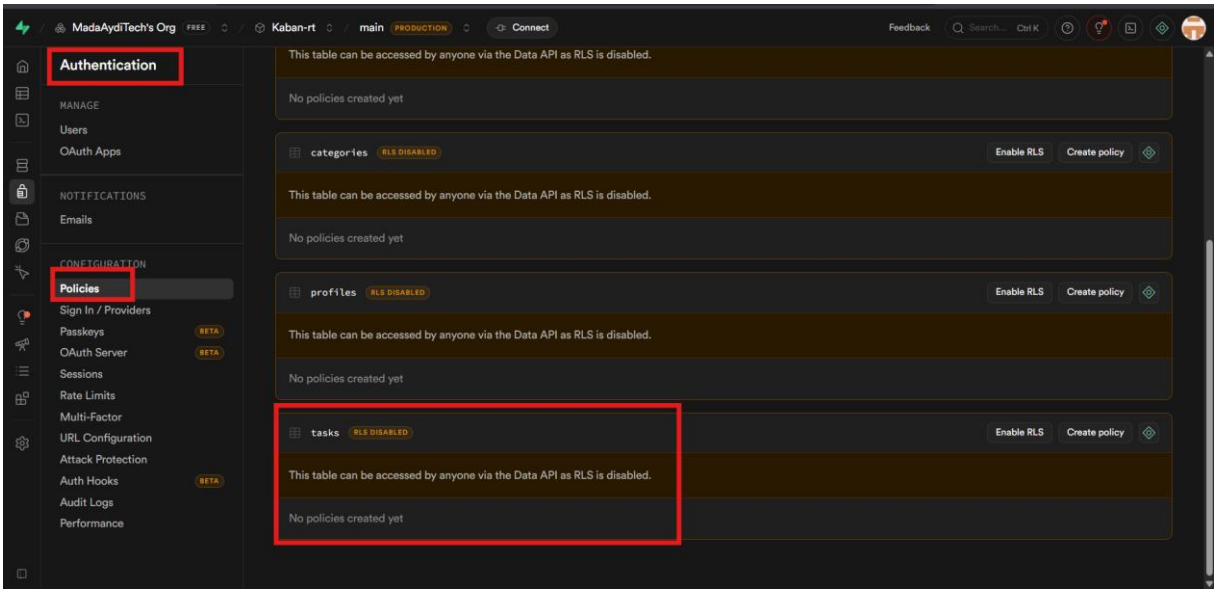


Difficultés & solutions :

Les premiers soucis sur lequel on est tombé c'était une tâche introuvable après la création (no rows found) :

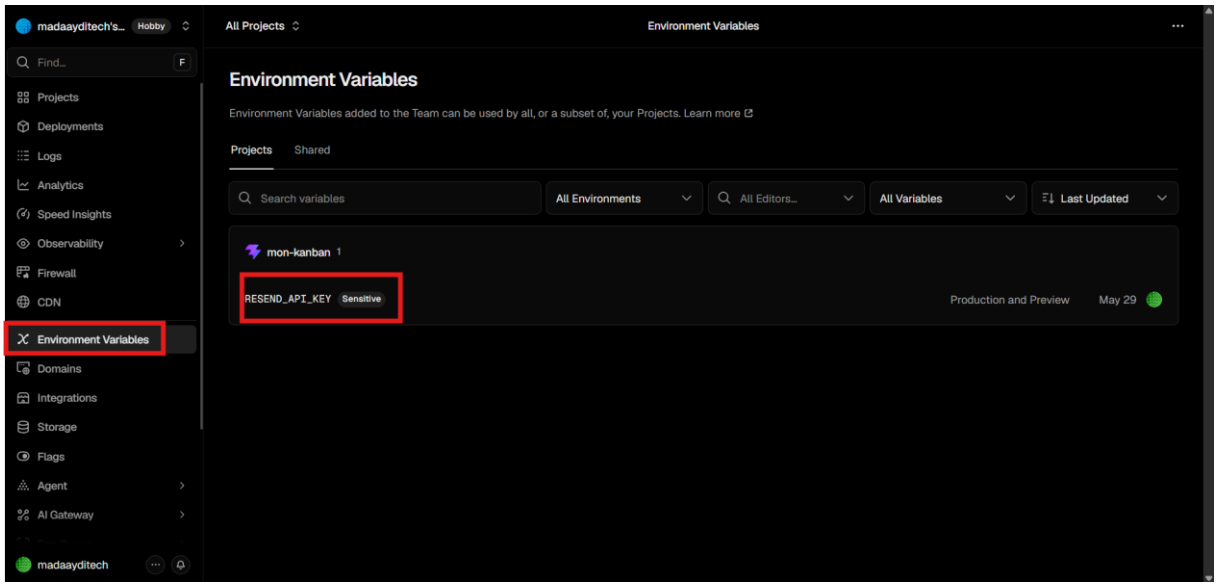
Après avoir crée des tâches avec SQL Editor, le composant TaskList affichait « Aucune tâche » malgré des lignes présentes en Base de données ça c'était due au Row Level Security (RLS) qui était activée par défaut sur la table Tasks et sans politique SELECT défini ce qui faisait que Supabase renvoyait un tableau vide pour tous les utilisateurs.

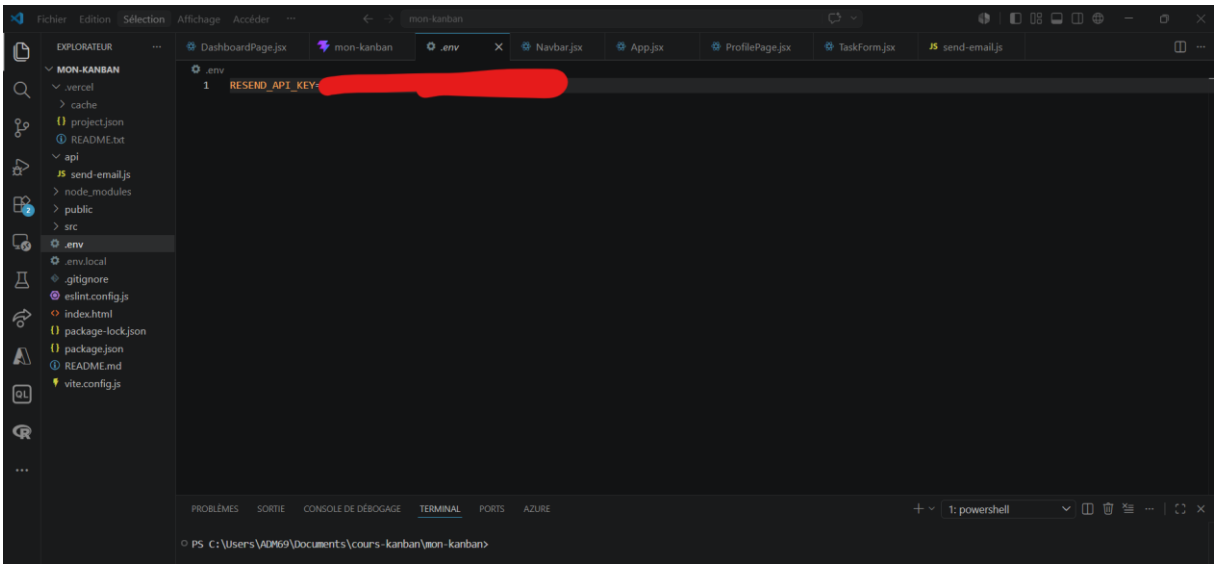
La solution qu'on a appliqué c'était de désactiver les temporaire au RLS sur la table tasks dans dans Authentification puis Policies :



Le second problème c'était au niveau des variables d'environnement, elles étaient absente en production de Vercel. L'application Vercel affichait une page blanche et sur le terminal c'était écrit

« supabaseUrl is required » et « supabaseKey is required ». Le fichier .env.local ne doit jamais être comité et Vercel ne connaissait pas les variables VITE_SUPABASE_URL et VITE_SUPABASE_ANON_KEY qui compose les clés. La solution ici c'était d'ajouter manuellement ses deux variables et de remplir les variables via des données qu'on trouve dans les paramètres de Vercel donc en allant sur Vercel puis sur Settings et ensuite Environment Variables, suivi d'un Redeploy dans l'onglet Deployments pour les prendre en compte.





Le dernier problème avec qui on a fait face c'était l'appel fetch ('/api/send-email') qui nous renvoyait une erreur 404 dans l'environnement local, c'était due à Vite qui ne prend pas en charge les dossiers serveless du dossier api qui sont des fonctions qui sont gérées que par Vercel. Afin d'y remédier on a installer (npm install -g vercel) et lancement de **Vercel dev** à la place de npm run dev pour lancer l'environnement Vercel complet en local.

Conclusion

Pour conclure, ce projet nous a permis de mettre en pratique l'ensemble de la chaîne de développement web moderne : création d'une SPA React avec Vite, intégration d'un backend avec Supabase, le travail collaboratif avec github et le déploiement continu sur Vercel, la fonctionnalité libre Pomodoro apporte une réelle valeur ajoutée à l'application**.**

Amélioration effectué (VOIR TRAVAIL DE MAXENCE) :

On a fait en sorte d'actualiser chaque tâche directement donc on a mis en place le Drag & drop entre colonnes pour permettre de glisser une carte d'une colonne à l'autre pour changer son statut directement, sans passer par un formulaire. **(voir travail de Maxence github maxence)**

On a même ajouté un filtre de recherche avancé on peut voir quels utilisateurs son présent sur la plateforme kanban.

Idée de proposition/d'amélioration :

On avait aussi pour idée d'ajouter un calendrier de suivie qui permet de voir l'évolution de l'utilisateur sa proactivité au sein du kanban mais on ne l'a pas fait finalement.

On avait une autre idée c'était de crée une application mobile mais on la pas fait.

On avait pour projet de réaliser un petit graphique un peu comme sur fifa mais nous dans le domaine scolaire on l'a pas fait… :



Outils de collaboration utilisé :

- **Github**
- **Liveshare (sur VSCODE)**

Nos réseaux collaboratifs/là ou on a travailler :

Adam ABIDERRAHMANE :

https://github.com/MadaAydiTech/mon-kanban

https://vercel.com/madaayditechs-projects

Maxence DABOIT :

https://github.com/maxdaboit/mon-kanban

mon-kanban-hk94q3vx7-maxdaboits-projects.vercel.app